

ARTIFICIAL INTELLIGENCE

Chapter 2: Intelligent Agents

Instructor: Dr Ghulam Mustafa

Department of Information Technology

PUGC

*In which we discuss the **nature of agents**, perfect or otherwise, **the diversity of environments**, and the resulting menagerie of **agent types**.*

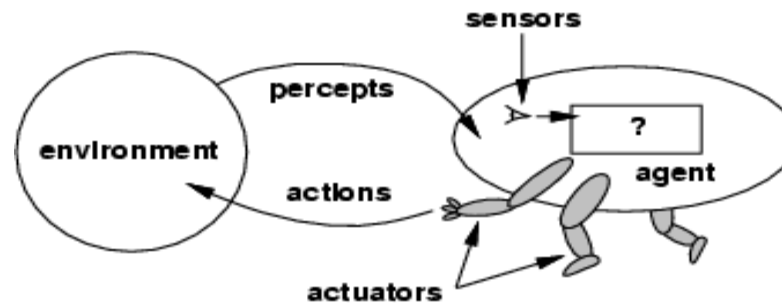
Outline

- Agents and environments
- Rationality
- PEAS (Performance measure, Environment, Actuators, Sensors)
- Environment types
- Agent types

Agents

- An **agent** is anything that can be viewed as **perceiving** its **environment** through **sensors** and **acting** upon that environment through **actuators**
- Human agent: eyes, ears, and other organs for sensors; hands, legs, mouth, and other body parts for actuators
- Robotic agent: cameras and infrared range finders for sensors; various motors for actuators
- Software agent: receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.

Agents and environments

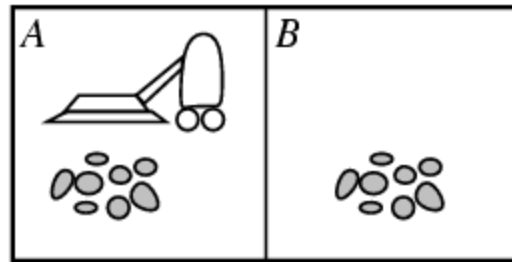


- The **agent function** maps from percept histories to actions:

$$[f: \mathcal{P}^* \rightarrow \mathcal{A}]$$

- The agent function implemented by an **agent program**
- The **agent program** runs on the physical **architecture** to produce f
- agent = architecture + program

Vacuum-cleaner world



- Percepts: location and contents, e.g., [A,Dirty]
- Actions: *Left*, *Right*, *Suck*, *NoOp*

A vacuum-cleaner agent

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

Rational agents

- An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform. The right action is the one that will cause the agent to be most successful
- **Performance measure:** An objective criterion for success of an agent's behavior. E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

Rational agents

- **Rational Agent:** For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has
- Example: vacuum-cleaner agent

Rational agents

- Rationality is distinct from omniscience (all-knowing with infinite knowledge)

Rational $\neq$ omniscient

- Agents can perform actions in order to modify future percepts so as to obtain useful information (information gathering, exploration)
- An agent is to **learn** as much as possible from what it perceives. — Dung beetle, Spheg wasp (no learning)
- An agent is **autonomous** if its behavior is determined by its own experience (with ability to learn and adapt)
- Rational $\Rightarrow$ exploration, learning, autonomy

Rationality

- Rationality is relative to performance measure
- Judge rationality based on
 - The performance measure that defines the criterion of success
 - The agent prior knowledge of the environment
 - The possible actions the agent can perform
 - The agent's percept sequence to date

PEAS

- PEAS: Performance measure, Environment, Actuators, Sensors
- Must first specify the setting for intelligent agent design
- Consider, e.g., the task of designing an automated taxi driver:
 - **Performance measure:** Safe, fast, legal, comfortable trip, maximize profits
 - **Environment:** Roads, other traffic, pedestrians, customers, weather
 - **Actuators:** Steering wheel, accelerator, brake, signal, horn
 - **Sensors:** Cameras, sonar, speedometer, GPS, odometer, engine sensors, keyboard

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Environment types

- **Fully observable** (vs. partially observable vs. unobservable): An agent's sensors give it access to the complete state of the environment at each point in time. E.g. Crossword puzzle
 - Agent need not maintain any internal state to keep track of the world.
- **Partially Observable**: an environment might be partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data
 - E.g. A vacuum agent with only a local dirt sensor cannot tell whether there is dirt in other squares, and an automated taxi cannot see what other drivers are thinking.
- If the agent has no sensors at all then the environment is **unobservable**.

Environment types

- **Single agent** (vs. multiagent): An agent solving a crossword puzzle by itself is clearly in a **single-agent** environment, whereas an agent playing chess is in a **two-agent** environment.
- Properties of Multi-Agent Environment:
 - Competitive (chess)
 - Cooperative (taxi driving (partially cooperative, partially competitive))
 - Communication – a rational behavior in multi-agent environment
 - Randomized behavior - avoids the pitfalls of predictability

Environment types

- **Deterministic** (vs. stochastic): The next state of the environment is completely determined by the current state and the action executed by the agent. (We ignore uncertainty that arises purely from the actions of other agents in a multiagent environment; thus, a game can be deterministic even though each agent may be unable to predict the actions of the others.)
- **Stochastic**: The next state can't be determined by current state. It generally implies that uncertainty about outcomes is quantified in terms of probabilities;
 - E.g. taxi driving
- An environment is **uncertain** if it is not fully observable or not deterministic.
- A **nondeterministic** environment is one in which actions are characterized by their *possible* outcomes, but no probabilities are attached to them.

Environment types

- **Episodic** (vs. sequential): The agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself. Many classification tasks are episodic.
- In **sequential environments**, the current decision could affect all future decisions. E.g. Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences.

Environment types

- **Static** (vs. dynamic): The environment is unchanged while an agent is deliberating. Crossword puzzles are static.
- **Semi-dynamic**: if the environment itself does not change with the passage of time but the agent's performance score does. E.g. Chess, when played with a clock.
- **Dynamic**: If the environment can change while an agent is deliberating; Taxi driving is dynamic: the other cars and the taxi itself keep moving while the driving algorithm dithers about what to do next.

Environment types

- **Discrete** (vs. continuous): A limited number of distinct, clearly defined percepts and actions. For example, the chess environment has a finite number of distinct states.
- Taxi driving is a continuous-state and continuous-time problem: the speed and location of the taxi and of the other vehicles sweep through a range of **continuous** values and do so smoothly over time.

Environment types

- **Known**(vs. unknown): this distinction refers not to the environment itself but to the agent's (or designer's) state of knowledge about the “laws of physics” of the environment. In a known environment, the outcomes for all actions are given.
- In **unknown** environment, the agent will have to learn how it works in order to make good decisions.
- Known/unknown is different from observable/ unobservable
- *Known* environment can be *partially observable*
 - *Solitaire card games. Know all rules but unable to see other cards*
- *Unknown* environment can be *fully observable*
 - *In video game, entire game state on screen but don't know about buttons*

Environment types

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

Agent functions and programs

- The job of AI is to design an **agent program** that implements the agent function
- This program will run on some sort of computing device with physical sensors and actuators called **architecture**

$$\textit{agent} = \textit{architecture} + \textit{program}$$

Agent functions and programs

- The program we choose has to be one that is appropriate for the architecture
- If program recommends actions like *Walk*, the architecture had better have legs.
- The architecture might be just an ordinary PC, or it might be a robotic car with several onboard computers, cameras, and other sensors.

Table-lookup agent

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Table-lookup agent

- $10^{250,000,000,000}$ entries for an hour's driving.
- At least 10^{150} entries for chess.
- Drawbacks:
 - Huge table
 - Take a long time to build the table
 - No autonomy
 - Even with learning, need a long time to learn

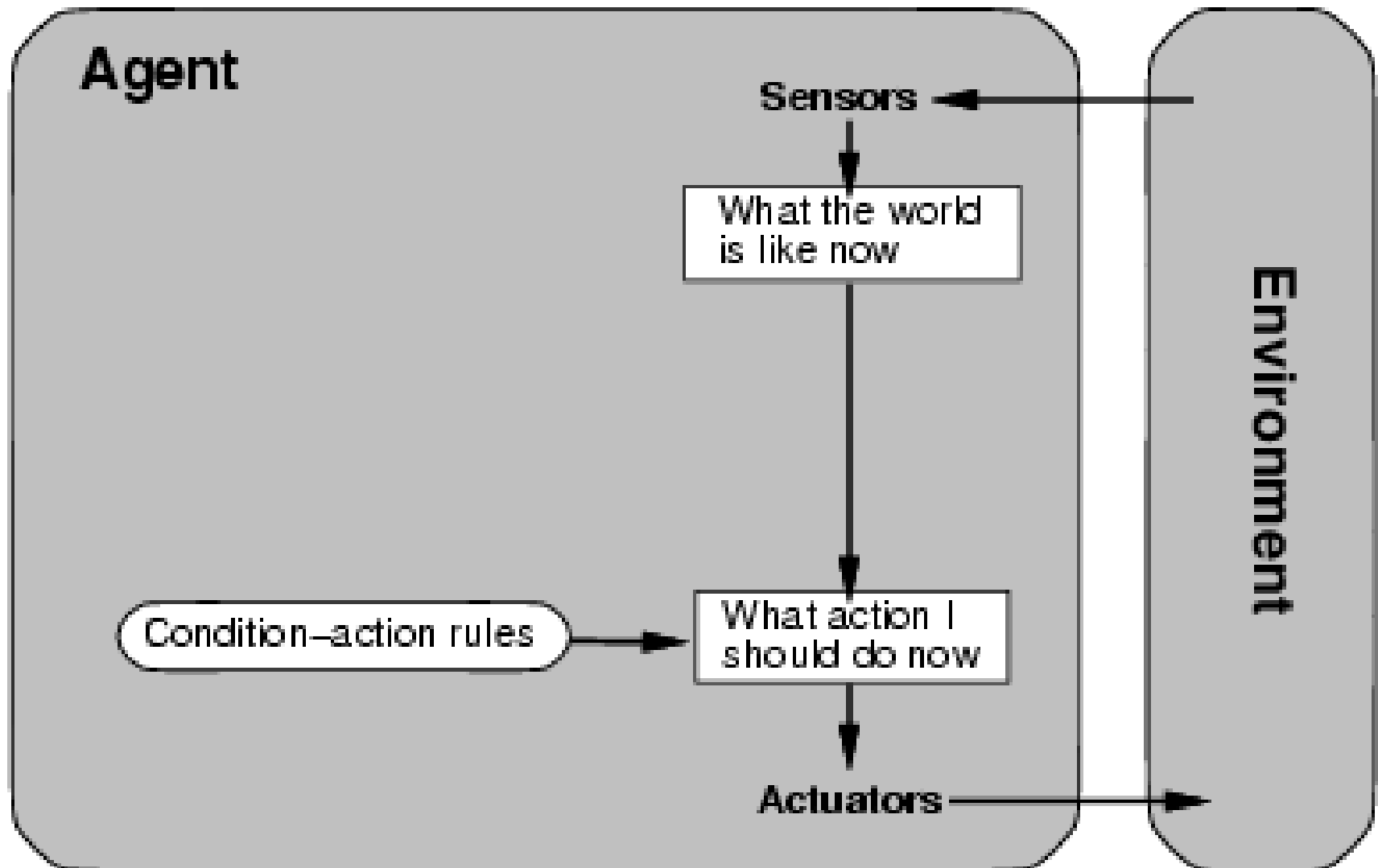
Agent types

- Four basic types in order of increasing generality:
- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents

Simple reflex agents

- Simple reflex agents use condition-action rule e.g.
 - **if** car-in-front-is-braking **then** initiate-braking
- Work best with fully observable environment
- Simple reflex agents are simple but limited intelligence
- *car-in-front-is-braking* works well only if there is centrally mounted light captured only in single frame of video
- Vacuum cleaner agent deprived from location sensor starts working in infinite loops
- Escape from infinite loops is possible if the agent can **randomize** its actions.

Simple reflex agents



Simple reflex agents

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action

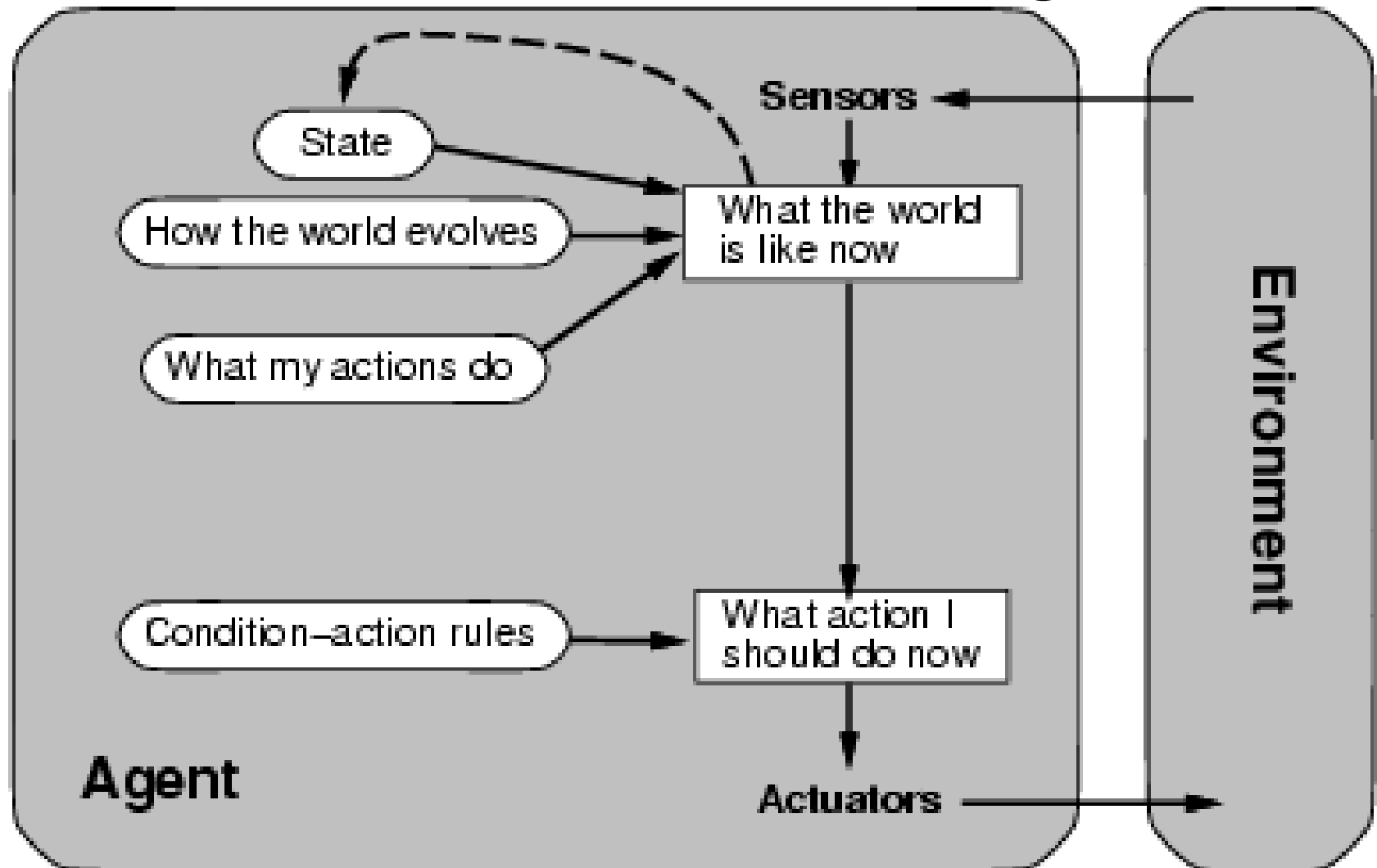
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

Model-based reflex agents

- Model-based agent maintains some sort of **internal state** that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state. Best for partial observability
- Two kind of knowledge is required to update the information about internal state
 - Some information about how the world evolves independently of the agent—for example, that an overtaking car generally will be closer behind than it was a moment ago.
 - Some information about how the agent's own actions affect the world—for example, that when the agent turns the steering wheel clockwise, the car turns to the right

Model-based reflex agents



Model-based reflex agents

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition–action rules
               action, the most recent action, initially none

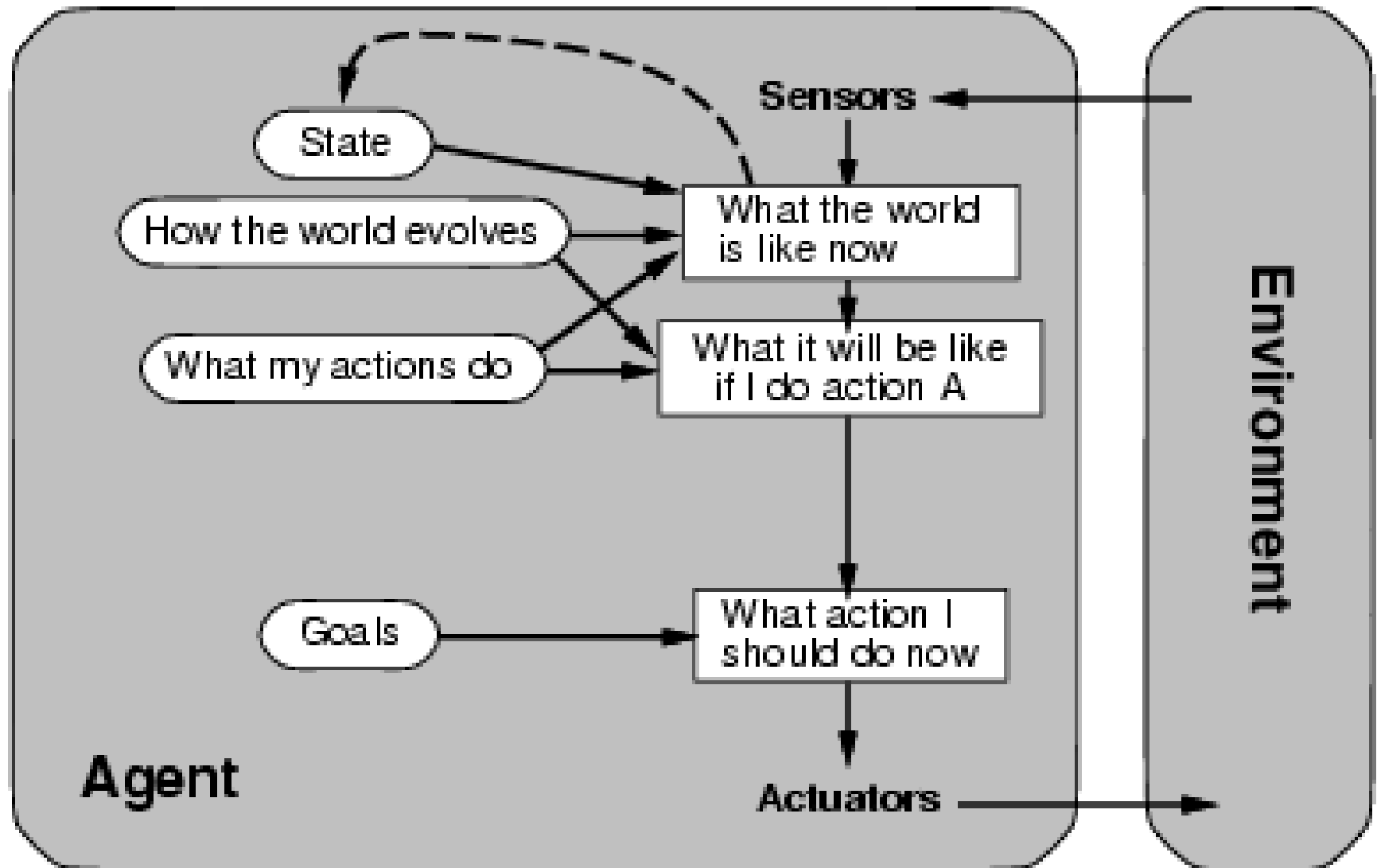
  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Goal-based agents

- Knowing the current state of the environment is not enough, needs some goal information
- This kind of agent has a goal and strategy to reach the goal
- All actions are based on its goal (not necessarily the best one)
- Example: searching robots that has initial location and want to reach a destination

Goal-based agents



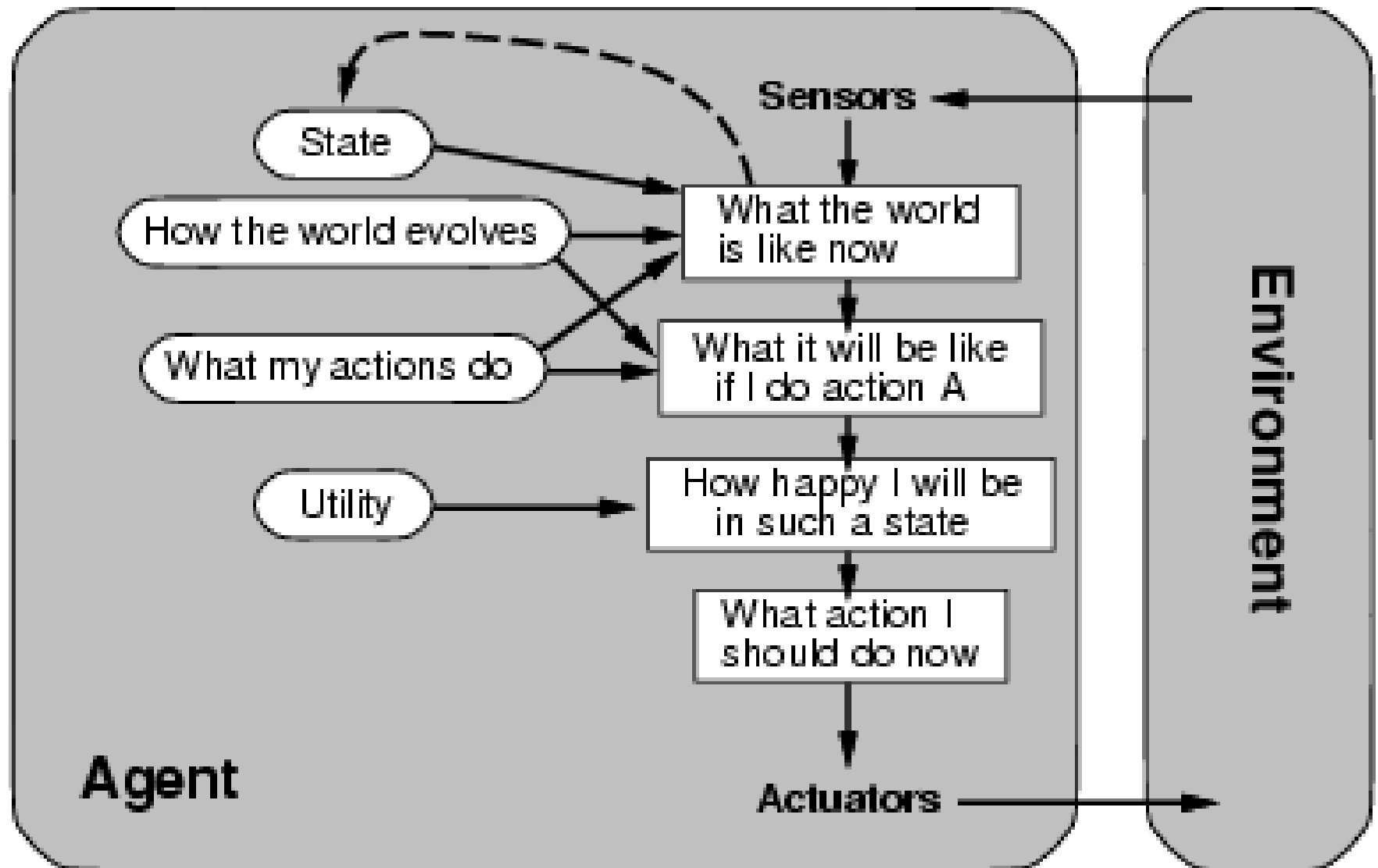
Utility-based agents

- Goals alone are not enough to generate high-quality behavior in most environments. For example, many action sequences will get the taxi to its destination (thereby achieving the goal) but some are quicker, safer, more reliable, or cheaper than others.
- A more general performance measure – how happy rather than happy/unhappy

Utility-based agents

- Goals are inadequate but a utility-based agent can still make rational decisions.
 - First, when there are conflicting goals, only some of which can be achieved (for example, speed and safety), the utility function specifies the appropriate tradeoff.
 - Second, when there are several goals that the agent can aim for, none of which can be achieved with certainty, utility provides a way in which the likelihood of success can be weighed against the importance of the goals.

Utility-based agents



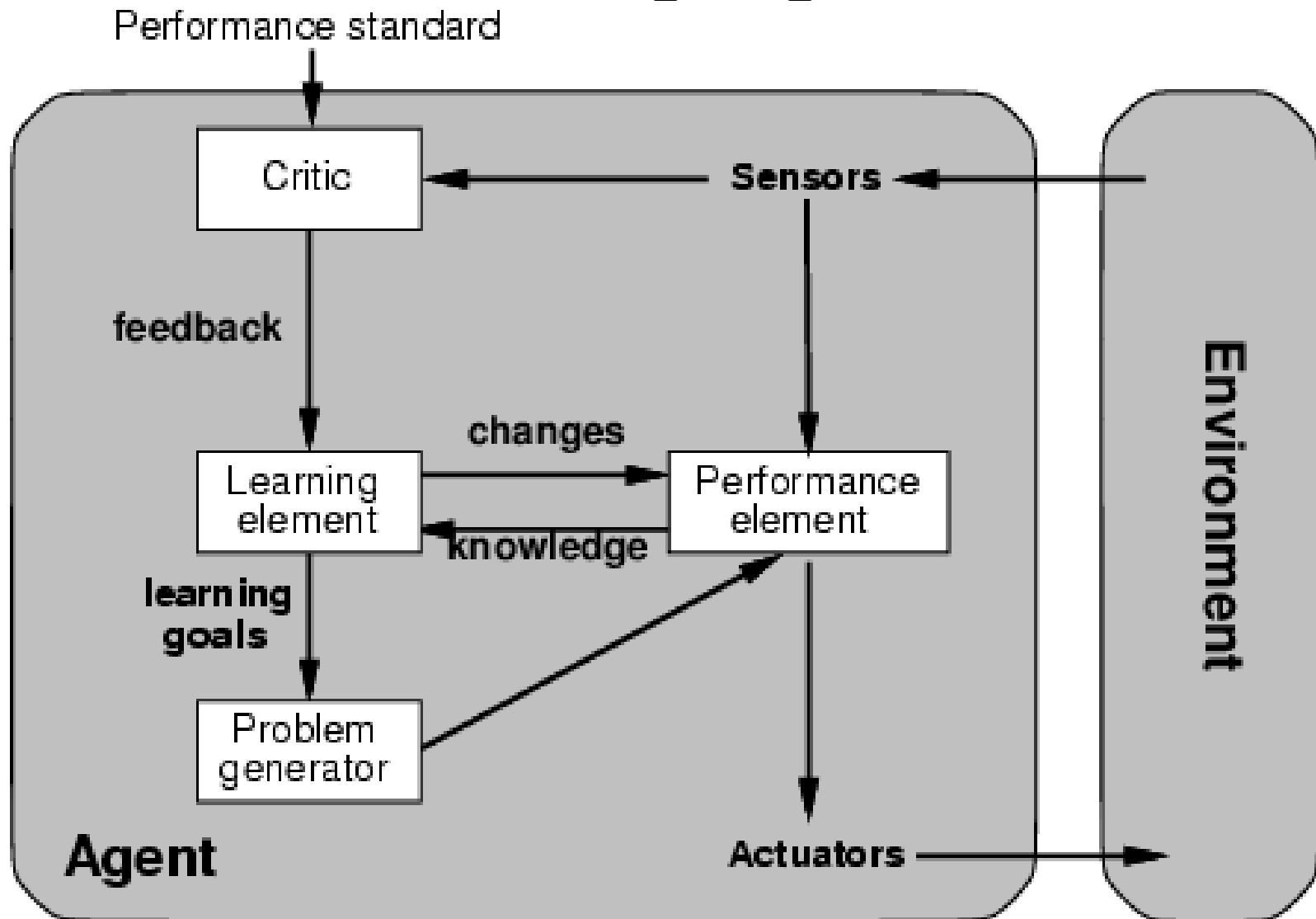
Learning agents

- The essential component of autonomy, this agent is capable of learning from experience
- It has the capability of automatic information acquisition and integration into the system
- Any agent designed and expected to be successful in an uncertain environment is considered to be learning agent

Learning agents

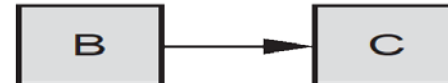
- Four conceptual components:
 - **Learning element**: responsible for making improvements
 - **Performance element**: responsible for selecting external actions. It is what we considered as agent so far
 - **Critic**: how well is the agent is doing w.r.t. a fixed performance standard
 - **Problem generator**: allows the agent to explore

Learning agents



Agent's organization

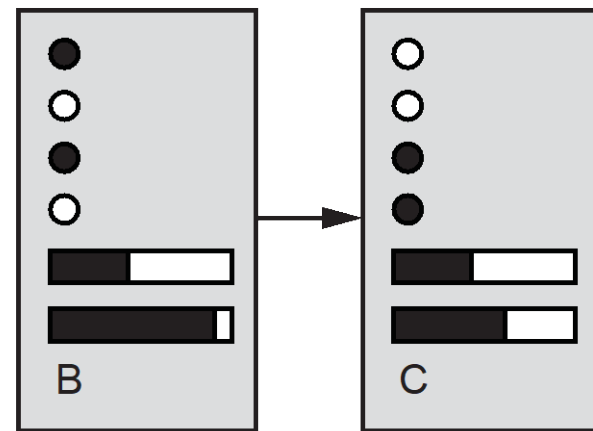
- **Atomic Representation:** Each state of the world is black-box that has no internal structure e.g. finding a driving route, each state is a city
- AI Algorithms: search, games, Markov decision processes, hidden Markov models, etc.



(a) Atomic

Agent's organization

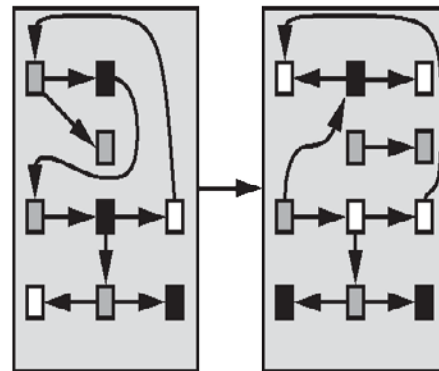
- **Factored Representation:** Each state has some attributes value properties e.g. GPS location, amount of gas in the tank
- AI Algorithms: constraint satisfaction and Bayesian networks



(b) Factored

Agent's organization

- **Structured Representation:** Relations between the objects of a state can be explicitly expressed.
- AI Algorithms: first order logic, knowledge based learning, natural language understanding



(b) Structured

Summary

- Agents interact with environments through actuators and sensors
- The agent function describes what the agent does in all circumstances
- The performance measure evaluates the environment sequence
- A perfectly rational agent maximizes expected performance
- Agent programs implement (some) agent functions
- PEAS descriptions define task environments
- Environments are categorized along several dimensions: observable? deterministic? episodic? static? discrete? single-agent?
- Several basic agent architectures exist: reflex, reflex with state, goal-based, utility-based